Environment variables

The backend reads settings through `backend/app/core/config.py` (Pydantic `BaseSettings`). Values come from the process environment, then from `backend/.env`. The env file path is **relative to the working directory**, so start `uvicorn`, `celery`, `alembic` and `pytest` from `backend/`. Names are case-insensitive. Copy `backend/.env.example` to `backend/.env`; never commit `.env`.

Required below means "the pipeline does not work without it", not "the process refuses to start" — most services are built lazily, so a missing value surfaces when first used.

Backend — required for a working local pipeline

Variable	Default	Purpose	Where to get it	If missing
`DATABASE_URL`	`postgresql+psycopg2://docauth:docauth@localhost:5432/docauth`	The **owner** connection — used only by Alembic and `seed.py` (needs CREATEROLE, not superuser). Its host/port/database are also used for the two app roles below	Matches the Postgres in `docker-compose.yml` (default works as-is)	Migrations fail; API/worker can't build their URLs
`DATABASE_APP_USER` / `DATABASE_APP_PASSWORD`	`fddt_app` / `fddt_app_local`	The role every company request and pipeline task connects as (NOBYPASSRLS — Row-Level Security applies). The migration creates it with this password	Choose a long random password **before** running `alembic upgrade head`	Every request fails to connect
`DATABASE_PLATFORM_USER` / `DATABASE_PLATFORM_PASSWORD`	`fddt_platform` / `fddt_platform_local`	The role for sign-in, platform-admin screens and usage reconciliation (all rows through the `platform_all` policy; no BYPASSRLS)	Same	Login and platform screens fail
`CELERY_BROKER_URL`	`redis://localhost:6379/0`	Celery broker	The Redis in `docker-compose.yml`	Uploads succeed but nothing is processed; documents stay `pending`
`CELERY_RESULT_BACKEND`	`redis://localhost:6379/1`	Celery results	Same Redis, DB 1	Worker errors on result storage
`JWT_SECRET_KEY`	`changeme-in-.env`	Signs login tokens (HS256)	Generate a long random string, e.g. `python -c "import secrets; print(secrets.token_urlsafe(48))"`	**Works with an insecure default — always change it**; changing it logs everyone out
`AZURE_STORAGE_CONNECTION_STRING`	*(none)*	Blob Storage account for originals, signature crops and reports	Azure portal → Storage account → *Access keys* → connection string	Upload and file-url endpoints return **503** with an explanatory message

Variable	Default	Purpose	Where to get it	If missing
`AZURE_STORAGE_CONTAINER_NAME`	`documents`	Blob container (must be **private**)	Create it in the storage account	Uploads fail
`AZURE_DOCUMENT_INTELLIGENCE_ENDPOINT`	*(none)*	OCR/layout (`prebuilt-layout`)	Azure portal → Document Intelligence resource → *Keys and Endpoint*	Every document ends `failed` at extraction
`AZURE_DOCUMENT_INTELLIGENCE_KEY`	*(none)*	Same	Same	Same
`AZURE_OPENAI_KEY`	*(none)*	Classification, extraction, issuer semantic match, visual review, signature detect/compare	Azure portal (or AI Foundry) → your OpenAI resource → *Keys*	Extraction fails (document `failed`); visual review/signature checks fail
`AZURE_OPENAI_ENDPOINT`	*(none)*	Resource endpoint. Accepts either `https://<res>.openai.azure.com/` or an AI Foundry project URL `https://<res>.services.ai.azure.com/api/projects/<name>` (normalized to the origin)	Same	Same
`AZURE_OPENAI_DEPLOYMENT_NAME`	*(none)*	The model **deployment** to call. **Must be vision-capable**: its name must contain `gpt-4o` , `gpt-4.1` , `gpt-4-turbo` or `gpt-4-vision` , or the visual review and signature features raise `LLMConfigurationError`	Name you gave the deployment in Azure	Same
`AZURE_OPENAI_MAX_REQUESTS_PER_MINUTE`	`60`	Global cap on Azure OpenAI requests across every worker (Redis limiter). Set to **~80 %** of the deployment's RPM	Azure portal → deployment → *Rate limit*, or the `x-ratelimit-limit-requests` response header	Default 60 may under- or over-shoot your quota (429s or idle capacity)
`AZURE_OPENAI_MAX_TOKENS_PER_MINUTE`	`0` (off)	Global token cap (prompt + `max_tokens` reservation per call). Usually the binding limit. Set to **~80 %** of the deployment's TPM	`x-ratelimit-limit-tokens` header / portal	Without it, bursts exceed TPM → 429s

There is **no local/offline substitute** for storage, OCR or the LLM (no Azurite/mock backend is wired in); a real Azure account is needed to exercise the pipeline. The unit tests mock all three, so they do not call Azure.

Backend — optional / tuning

Variable	Default	Purpose
`ENVIRONMENT`	`local`	Label shown in `/health` and the OpenAPI description
`JWT_ALGORITHM`	`HS256`	JWT algorithm

Variable	Default	Purpose
`JWT_ACCESS_TOKEN_EXPIRE_MINUTES`	`480` (8 h)	Login token lifetime
`AZURE_OPENAI_API_VERSION`	`2024-10-21`	Azure OpenAI REST API version
`AZURE_OPENAI_REQUEST_TIMEOUT_SECONDS`	`60.0`	Per-call LLM timeout. Kept short on purpose: a hung call holds a worker thread (and its rate-limiter reservation)
`AZURE_DOCUMENT_INTELLIGENCE_MAX_CALLS_PER_SECOND`	`10.0`	Global cap on Document Intelligence calls (Azure's default S0 limit is 15 TPS)
`DATABASE_POOLER_HOST` / `DATABASE_POOLER_PORT`	*(unset)* / `6432`	When set, the two app roles connect through **PgBouncer** (transaction pooling) and each process keeps no pool of its own. Recommended in production (processing-queues.md)
`DB_POOL_SIZE` / `DB_MAX_OVERFLOW`	`5` / `10`	SQLAlchemy pool per role per process when connecting **directly** (no PgBouncer)
`DEFAULT_MAX_FILE_SIZE_MB` / `DEFAULT_MAX_ZIP_SIZE_MB`	`10` / `300`	Upload limits a **new** company is created with. Limits are per company (`companies.max_file_size_mb` / `max_zip_size_mb` , changed by a platform admin under Platform › Companies); changing these defaults never changes an existing company. The frontend reads the company's real limits from `GET /auth/me/upload-limits`
`BULK_UPLOAD_MAX_ENTRIES`	`25000`	Safety cap on entries (files + folders) in one zip. A 300 MB zip of small born-digital PDFs holds about 11,000 entries
`BULK_UPLOAD_CASE_WARNING_THRESHOLD`	`100`	Above this many cases the UI warns; the zip is still accepted
`DEFAULT_COMPANY_NAME`	`Default Company`	Name of the company the multi-tenancy migration assigns pre-existing data to (used once, by the migration)
`PLATFORM_ACCESS_AUDIT_DEDUP_SECONDS`	`0`	0 = every platform-admin read of company data writes its own (platform-only) audit row; > 0 collapses identical reads within that window
`EXTRACTION_WORKER_CONCURRENCY` / `VISION_WORKER_CONCURRENCY`	`4` / `4`	Threads per extraction / vision worker (`start-worker.sh`). Size to just saturate the Azure caps
`FORENSICS_WORKER_CONCURRENCY`	`0`	Prefork processes per forensics worker; 0 = one per CPU of the container's limit
`FAIR_SHARE_BUCKET_SIZE`	`5`	A company's tasks drop one priority level per this many outstanding tasks on a queue

Variable	Default	Purpose
`QUEUE_ALERT_OLDEST_WAITING_SECONDS`	`300`	The per-minute `queue_metrics` job logs a WARNING `queue_alert {json}` line when a queue's oldest waiting task is older than this; 0 = off
`USAGE_RECONCILIATION_HOUR_UTC`	`2`	Hour (UTC) of the nightly usage-counter reconciliation (Celery beat)
`SEED_ADMIN_EMAIL` / `SEED_ADMIN_PASSWORD`	`admin@example.com` / `ChangeMe123!`	Read by `seed.py` (not by the app): the platform admin it creates — **and resets on every run** (`start-api.sh` runs it). Set both in production
`CELERY_POOL`	per queue	Read by `start-worker.sh` : overrides the pool type (`threads` for extraction/vision, `prefork` for forensics)
`CELERY_BEAT_SCHEDULE_FILE`	`/tmp/celerybeat-schedule`	Read by `start-beat.sh`
`ISSUER_FUZZY_MATCH_THRESHOLD`	`85.0`	rapidfuzz WRatio score (0–100) an extracted issuer must reach to match the registry (05)
`METADATA_FORENSICS_MOD_DATE_THRESHOLD_SECONDS`	`1.0`	Jitter tolerance before "modified after created" flags (06)
`METADATA_FORENSICS_EDITING_SOFTWARE_NAMES`	`photoshop,gimp,illustrator,affinity,coreldraw,paint.net,pixlr,canva,inkscape`	Comma-separated, case-insensitive substrings that mark a Producer/Creator as an editing tool
`DUPLICATE_HASH_HAMMING_THRESHOLD`	`5`	Max pHash Hamming distance (0–64) for a near-duplicate (10)

Risk **weights, severities and tier thresholds are not environment variables** — they live in the database, per company, and are edited in Settings › Risk Rules (and the platform template under Platform › Rule templates; see 11).

Backend — declared but UNUSED (do not need a value)

Variable	Would be for	Status
`AZURE_AI_VISION_ENDPOINT` , `AZURE_AI_VISION_KEY`	Signature/stamp detection, logo matching	Never read — detection uses the Azure OpenAI vision model
`AI_CONTENT_DETECTION_ENDPOINT` , `AI_CONTENT_DETECTION_KEY`	Content Safety / Hive	Never read — see 08
`EMAIL_SERVICE_CONNECTION_STRING` , `EMAIL_FROM_ADDRESS`	Transactional email	Never read — notifications are not built
`API_V1_PREFIX` (`/api/v1`)	URL prefix	Never read — routes have no prefix
`REDIS_URL`	Redis	Never read — Celery, the rate limiter, the fair-share counters and the queue monitor all use `CELERY_BROKER_URL`

Variable	Would be for	Status
`ANTHROPIC_API_KEY`, `ANTHROPIC_MODEL`	Claude API	Never read — the `LLMService` implementation in use is Azure OpenAI

docker-compose (optional overrides)

Read by `docker-compose.yml` from the shell or a root `.env`; defaults shown.

Variable	Default	Purpose
`POSTGRES_USER` / `POSTGRES_PASSWORD` / `POSTGRES_DB`	`docauth` / `docauth` / `docauth`	Postgres container credentials. If you change them, update `DATABASE_URL` to match
`POSTGRES_PORT`	`5432`	Host port
`REDIS_PORT`	`6379`	Host port
`PGBOUNCER_PORT`	`6432`	Host port of PgBouncer (`pgbouncer/pgbouncer.ini`, `pgbouncer/userlist.txt` — dev passwords only)
`FORENSICS_WORKER_CPUS`	`2`	CPUs per `forensics-worker` replica (`--profile workers`)

Frontend

Variable	Default	Purpose
`VITE_API_BASE_URL`	`/api`	Base URL the SPA calls (build time). The default relies on a proxy that maps `/api/*` → the API with the prefix stripped — the Vite dev server locally, the ingress in production (production-deployment.md). Setting it to a different origin instead requires CORS on the backend, which currently has **no** CORS middleware

No other frontend variables exist. Login state is a JWT in `localStorage`.

Secrets checklist

`AZURE_STORAGE_CONNECTION_STRING`, `AZURE_DOCUMENT_INTELLIGENCE_KEY`, `AZURE_OPENAI_KEY`, `JWT_SECRET_KEY`, the owner database password (in `DATABASE_URL`), `DATABASE_APP_PASSWORD`, `DATABASE_PLATFORM_PASSWORD` and `SEED_ADMIN_PASSWORD` are the secrets (plus PgBouncer's `userlist.txt`). Keep them out of the repo (`.env` is git-ignored) and, on Azure, in Key Vault referenced from the container apps (see production-deployment.md).